Programming Fundamental Project

ECAT Online Examination Portal

Project Report:

Name:	Ehtisham Ahmed.
Roll No:	2026(S)-CYS-95.
Submitted To:	Mr. Mubashir.
Date:	9th June, 2026
Program:	Cyber Security.
Department:	Computer Engineering.

- **Problem Statement:**

Taking exams by hand has always been a headache for both teachers and students. Every time an exam comes around, the teacher has to write out questions, go through every paper to mark answers, add up scores, and somehow keep all that data in one place without making mistakes. It sounds simple enough, but even with a small class it takes hours, and errors creep in pretty easily.

I ran into this problem while thinking about how ECAT preparation is usually handled, and it got me wondering — why not just build something that takes care of all this? That thought is basically where this project started. The idea was to write a Python-based portal where a teacher can set up an exam, students sit it on a computer, and results come out automatically without anyone having to mark a single paper by hand.

- **Objective:**

1. Build a working online exam system for ECAT-style multiple-choice tests.
2. Have the program calculate scores, percentages, and grades on its own so there is no room for marking errors.
3. Give the admin enough control to add or remove questions whenever needed, without touching the code directly.
4. Keep the portal secure so students can only access their own exam and the admin side is password protected.

- **Methodology:**

I wrote everything in Python since that is what we have been studying this semester and I wanted to actually use what I have learned. The program is split into two parts — an Admin Portal and a Student Portal. Both ask for a password when you open them, so nobody can just walk in and mess with the questions or look at results they should not see.

For the question bank I used a list of dictionaries, which felt like the most natural fit for this kind of data. Each dictionary holds the subject, the question itself, four options labeled A through D, and the correct answer. The admin can add a new question by just typing it in, or delete one that is outdated. It was a good exercise in working with nested data structures.

Once a student logs in, they get a quick look at the rules and marking scheme before the exam starts. I added that because I know from experience that finding out there is negative marking after you have already guessed on half the paper is not fun. During the exam they can move forward, go back, skip a question, or submit early. Every answer gets saved in a temporary list so nothing is lost before they hit submit.

After submitting, the program goes through each answer and compares it to the correct answer in the question bank. Right answer adds marks, wrong answer takes some off as a penalty, and a skipped question gets zero — same rules as the real ECAT. Once that is done, the total, percentage, and a letter grade are calculated and stored right away.

The admin can see a list of everyone who sat the exam, or search by roll number to check how a specific student did. I also added a small stats section that shows the highest score, the lowest, and the class average. I thought it would be useful for a teacher to quickly see whether the exam was too hard or too easy.

- **Results:**

I tested the portal several times with different inputs to make sure everything worked properly. Here is what came out fine:

i. Login worked correctly for both admin and student accounts.
ii. Adding, viewing, and deleting questions from the admin panel worked as expected.
iii. The exam ran smoothly with question navigation — forward, back, and skip all functioned properly.
iv. Scores, percentages, and grades were calculated correctly after each submission.
v. Results could be viewed any time during the same program session.

vi. Class statistics — highest, lowest, and average — were correct after multiple students completed the exam.

I also tested some edge cases, like submitting without answering anything, and changing an answer before submitting. Both handled fine. The main limitation is that results are lost when the program closes since I used in-memory storage, but that was fine for this project.

- ## Conclusion

This project ended up teaching me more than I expected. I thought it would mainly be about writing loops and conditions, but I also had to think about how to structure the whole program properly, handle user input without things breaking, and make sure both portals worked independently. Working through those things was honestly the most useful part.

The portal does what it was supposed to do — exams run, papers get marked, results come out. If I keep working on it later, the first thing I would add is proper database storage so results are not lost when the program closes. A simple graphical interface would also help make it easier to use. For a first semester project, though, I am happy with how it turned out.